trainex2.py

```python
import sys, time, subprocess, gc
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_curve, auc,
    precision_recall_fscore_support, f1_score, recall_score,
)
from sklearn.preprocessing import label_binarize

from config2 import (
    seed_everything, SEED, DEVICE, OUTPUT_DIR, FIGURES_DIR,
    NUM_CLASSES, CLASS_NAMES, IMAGE_SIZE, IN_CHANNELS,
    BATCH_SIZE, GRAD_ACCUM_STEPS, EPOCHS, PATIENCE, VERSIONS,
    BEST_MODEL_PATH, BEST_MODEL_PATH2, UTIL_SAVE_PATH,
    version_model_path, version_history_path,
)
from dataset2 import get_dataloaders, GrayImageDataset
from model2 import SamVGG16Ex2
from torch.utils.data import DataLoader


def fmt_time(s): m, s = divmod(int(s), 60); return f"{m}m {s:02d}s" if m else f"{s}s"


# ── MixUp ──────────────────────────────────────────────────────────────
def mixup_batch(x, y, alpha, device):
    lam = float(np.random.beta(alpha, alpha)) if alpha > 0 else 1.0
    idx = torch.randperm(x.size(0), device=device)
    return lam * x + (1 - lam) * x[idx], y, y[idx], lam


# ── Train / validate one epoch ─────────────────────────────────────────
def train_one_epoch(model, loader, criterion, optimizer, scaler, device,
                    use_amp, accum_steps, use_mixup, mixup_alpha):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    optimizer.zero_grad(set_to_none=True)

    for step, (images, labels) in enumerate(loader):
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        with torch.amp.autocast(device.type, enabled=use_amp):
            if use_mixup:
                mixed, ya, yb, lam = mixup_batch(images, labels, mixup_alpha, device)
                logits = model(mixed)
                loss = lam * criterion(logits, ya) + (1 - lam) * criterion(logits, yb)
            else:
                logits = model(images)
                loss = criterion(logits, labels)
            loss = loss / accum_steps

        scaler.scale(loss).backward()
        if (step + 1) % accum_steps == 0 or (step + 1) == len(loader):
            scaler.step(optimizer)
            scaler.update()
            optimizer.zero_grad(set_to_none=True)

        running_loss += loss.item() * accum_steps * images.size(0)
        _, pred = logits.max(1)
        total += labels.size(0)
        correct += pred.eq(labels).sum().item()

    return running_loss / total, 100.0 * correct / total
```

```python


@torch.no_grad()
def validate(model, loader, criterion, device, use_amp):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0
    all_y, all_p, all_prob = [], [], []
    for images, labels in loader:
        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)
        with torch.amp.autocast(device.type, enabled=use_amp):
            logits = model(images)
            loss = criterion(logits, labels)
        prob = torch.softmax(logits.float(), dim=1)
        _, pred = logits.max(1)
        running_loss += loss.item() * images.size(0)
        total += labels.size(0); correct += pred.eq(labels).sum().item()
        all_y.append(labels.cpu().numpy()); all_p.append(pred.cpu().numpy())
        all_prob.append(prob.cpu().numpy())
    return (running_loss / total, 100.0 * correct / total,
            np.concatenate(all_y), np.concatenate(all_p), np.vstack(all_prob))


# ── Train a single version ──────────────────────────────────────────
def train_version(cfg, train_df, val_df):
    seed_everything(SEED)
    name = cfg["name"]
    train_loader, val_loader, eval_tf = get_dataloaders(
        train_df, val_df, aug_level=cfg["aug_level"], batch_size=BATCH_SIZE)

    model = SamVGG16Ex2(num_classes=NUM_CLASSES, in_channels=IN_CHANNELS,
                        dropout=cfg["dropout"]).to(DEVICE)
    total_p, train_p = model.count_parameters()

    counts = np.bincount(train_df["label"].values, minlength=NUM_CLASSES).astype(np.float64)
    w = (1.0 / (counts + 1e-8)); w = w / w.sum() * NUM_CLASSES
    weight = torch.tensor(w, dtype=torch.float32, device=DEVICE)
    criterion = nn.CrossEntropyLoss(weight=weight, label_smoothing=cfg["label_smoothing"])

    optimizer = torch.optim.AdamW(model.parameters(),
                                  lr=cfg["lr"], weight_decay=cfg["weight_decay"])
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode="max", factor=0.5, patience=3)   # NOTE: no 'verbose' (API change)
    use_amp = (DEVICE.type == "cuda")
    scaler  = torch.amp.GradScaler(DEVICE.type, enabled=use_amp)

    gpu = torch.cuda.get_device_name(0) if use_amp else "CPU"
    print("\n" + "=" * 92)
    print(f"  TRAIN [{name}] | aug={cfg['aug_level']} lr={cfg['lr']} wd={cfg['weight_decay']} "
          f"mixup={cfg['use_mixup']} ls={cfg['label_smoothing']}")
    print(f"  Device {gpu} | params {total_p:,} | batch {BATCH_SIZE}x{GRAD_ACCUM_STEPS} "
          f"| {len(train_df)} train / {len(val_df)} val")
    print("=" * 92)

    best_f1, patience_ctr, history = 0.0, 0, []
    t0 = time.time()
    for epoch in range(1, EPOCHS + 1):
        te = time.time()
        tr_loss, tr_acc = train_one_epoch(
            model, train_loader, criterion, optimizer, scaler, DEVICE,
            use_amp, GRAD_ACCUM_STEPS, cfg["use_mixup"], cfg["mixup_alpha"])
        vl_loss, vl_acc, vy, vp, _ = validate(model, val_loader, criterion, DEVICE, use_amp)
        vl_f1 = f1_score(vy, vp, average="macro", zero_division=0)
        scheduler.step(vl_f1)

        et = time.time() - te
        marker = ""
        if vl_f1 > best_f1:
            best_f1, patience_ctr = vl_f1, 0
            torch.save({"epoch": epoch, "model_state": model.state_dict(),
                        "val_acc": vl_acc, "val_f1_macro": vl_f1,
                        "class_names": CLASS_NAMES, "image_size": IMAGE_SIZE,
```

```
                        "in_channels": IN_CHANNELS, "dropout": cfg["dropout"],
                        "seed": SEED, "version": name},
                       version_model_path(name))
            marker = " ★ Best"
        else:
            patience_ctr += 1

        history.append({"epoch": epoch, "train_loss": tr_loss, "train_acc": tr_acc,
                        "val_loss": vl_loss, "val_acc": vl_acc, "val_f1": vl_f1,
                        "lr": optimizer.param_groups[0]["lr"], "time_s": et})
        pd.DataFrame(history).to_csv(version_history_path(name), index=False)

        print(f"  Ep {epoch:>3d}/{EPOCHS} | Tr L {tr_loss:.3f} A {tr_acc:5.2f}% | "
              f"Val L {vl_loss:.3f} A {vl_acc:5.2f}% F1 {vl_f1:.3f} | "
              f"LR {optimizer.param_groups[0]['lr']:.1e} | ⏱{fmt_time(et)} "
              f"ETA {fmt_time(et*(EPOCHS-epoch))}{marker}")
        if patience_ctr >= PATIENCE:
            print(f"  Early stop @ epoch {epoch} (no val-F1 gain for {PATIENCE}).")
            break

    print(f"  [{name}] done in {fmt_time(time.time()-t0)} | best val macro-F1 {best_f1:.4f}")

    # reload best for evaluation/benchmark
    ckpt = torch.load(version_model_path(name), map_location=DEVICE, weights_only=False)
    model.load_state_dict(ckpt["model_state"])
    hist = pd.DataFrame(history)
    return model, hist, eval_tf, (total_p, train_p)


# ── Inference benchmark (speed + memory) ──────────────────────────────
@torch.no_grad()
def benchmark(model, eval_tf, val_df, total_p):
    model.eval()
    ds = GrayImageDataset(val_df, transform=eval_tf)
    loader = DataLoader(ds, batch_size=BATCH_SIZE, shuffle=False)
    use_amp = (DEVICE.type == "cuda")
    if DEVICE.type == "cuda":
        torch.cuda.reset_peak_memory_stats(); torch.cuda.synchronize()
    # warmup
    for images, _ in loader:
        with torch.amp.autocast(DEVICE.type, enabled=use_amp):
            model(images.to(DEVICE)); break
    if DEVICE.type == "cuda": torch.cuda.synchronize()
    t0 = time.time(); n = 0
    for images, _ in loader:
        with torch.amp.autocast(DEVICE.type, enabled=use_amp):
            model(images.to(DEVICE, non_blocking=True))
        n += images.size(0)
    if DEVICE.type == "cuda": torch.cuda.synchronize()
    dt = time.time() - t0
    peak_mb = (torch.cuda.max_memory_allocated() / 1e6) if DEVICE.type == "cuda" else 0.0
    return {"ms_per_image": dt / n * 1000, "img_per_sec": n / dt,
            "peak_vram_mb": peak_mb, "params_million": total_p / 1e6,
            "size_mb": total_p * 4 / 1e6}


# ── Plots ─────────────────────────────────────────────────────────────
def plot_learning_curves(hist, name):
    fig, (a, b) = plt.subplots(1, 2, figsize=(14, 5))
    a.plot(hist["epoch"], hist["train_loss"], "o-", label="Train")
    a.plot(hist["epoch"], hist["val_loss"], "o-", label="Val")
    a.set_title(f"Loss [{name}]"); a.set_xlabel("Epoch"); a.legend(); a.grid(alpha=.3)
    b.plot(hist["epoch"], hist["train_acc"], "o-", label="Train Acc")
    b.plot(hist["epoch"], hist["val_acc"], "o-", label="Val Acc", color="green")
    b.plot(hist["epoch"], hist["val_f1"]*100, "s--", label="Val F1x100", color="purple")
    b.set_title(f"Accuracy / F1 [{name}]"); b.set_xlabel("Epoch"); b.legend(); b.grid(alpha=.3)
    plt.tight_layout(); p = FIGURES_DIR / f"learning_curves_{name}.png"
    plt.savefig(p, dpi=150, bbox_inches="tight"); plt.close()
    print(f"  ✓ {p}")

def plot_cm(y, p, name):
    cm = confusion_matrix(y, p)
```

```
    for norm, cmap, sfx, fmt in [(False, "Blues", "", "d"), (True, "Oranges", "_norm", ".1f")]:
        m = cm.astype(float) / cm.sum(axis=1, keepdims=True) * 100 if norm else cm
        plt.figure(figsize=(7, 6))
        sns.heatmap(m, annot=True, fmt=fmt, cmap=cmap,
                    xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES)
        plt.xlabel("Predicted"); plt.ylabel("Truth")
        plt.title(f"Confusion Matrix{' (%)' if norm else ''} [{name}]")
        plt.tight_layout(); pth = FIGURES_DIR / f"cm_{name}{sfx}.png"
        plt.savefig(pth, dpi=150, bbox_inches="tight"); plt.close(); print(f"  ✓ {pth}")

def plot_roc(y, prob, name):
    yb = label_binarize(y, classes=list(range(NUM_CLASSES)))
    plt.figure(figsize=(7, 6)); colors = plt.cm.Set1.colors
    for i, c in enumerate(CLASS_NAMES):
        fpr, tpr, _ = roc_curve(yb[:, i], prob[:, i])
        plt.plot(fpr, tpr, color=colors[i % len(colors)], lw=2,
                 label=f"{c} (AUC={auc(fpr,tpr):.3f})")
    plt.plot([0, 1], [0, 1], "k--", alpha=.5); plt.legend(loc="lower right")
    plt.xlabel("FPR"); plt.ylabel("TPR"); plt.title(f"ROC OvR [{name}]")
    plt.tight_layout(); p = FIGURES_DIR / f"roc_{name}.png"
    plt.savefig(p, dpi=150, bbox_inches="tight"); plt.close(); print(f"  ✓ {p}")

def plot_per_class(y, p, name):
    pr, rc, f1, _ = precision_recall_fscore_support(
        y, p, labels=list(range(NUM_CLASSES)), zero_division=0)
    x = np.arange(NUM_CLASSES); w = .25
    plt.figure(figsize=(10, 5))
    plt.bar(x - w, pr, w, label="Precision"); plt.bar(x, rc, w, label="Recall")
    plt.bar(x + w, f1, w, label="F1")
    plt.xticks(x, CLASS_NAMES); plt.ylim(0, 1.15); plt.legend(); plt.grid(axis="y", alpha=.3)
    plt.title(f"Per-class P/R/F1 [{name}]"); plt.tight_layout()
    p_ = FIGURES_DIR / f"per_class_{name}.png"
    plt.savefig(p_, dpi=150, bbox_inches="tight"); plt.close(); print(f"  ✓ {p_}")


# ── Evaluate one version fully ──────────────────────────────────────
def evaluate_version(model, eval_tf, train_df, val_df, hist, name, bench):
    use_amp = (DEVICE.type == "cuda"); crit = nn.CrossEntropyLoss()
    # clean train loader (for overfit gap)
    tr_loader = DataLoader(GrayImageDataset(train_df, transform=eval_tf),
                           batch_size=BATCH_SIZE, shuffle=False)
    val_loader = DataLoader(GrayImageDataset(val_df, transform=eval_tf),
                            batch_size=BATCH_SIZE, shuffle=False)
    _, tr_acc, *_ = validate(model, tr_loader, crit, DEVICE, use_amp)
    _, vl_acc, vy, vp, vprob = validate(model, val_loader, crit, DEVICE, use_amp)

    vl_f1   = f1_score(vy, vp, average="macro", zero_division=0)
    per_rec = recall_score(vy, vp, labels=list(range(NUM_CLASSES)),
                           average=None, zero_division=0)
    gap = tr_acc - vl_acc

    print(f"\n  ── Eval [{name}] ── Val Acc {vl_acc:.2f}% | macro-F1 {vl_f1:.4f} | "
          f"gap {gap:.2f}% | min class-recall {per_rec.min():.3f}")
    print(classification_report(vy, vp, target_names=CLASS_NAMES, zero_division=0))

    plot_learning_curves(hist, name)
    plot_cm(vy, vp, name); plot_roc(vy, vprob, name); plot_per_class(vy, vp, name)

    # selection score: reward F1, punish overfitting gap and class collapse
    overfit_pen   = max(0.0, gap - 8.0) / 100.0
    collapse_pen  = max(0.0, 0.80 - per_rec.min()) * 0.5
    score = vl_f1 - overfit_pen - collapse_pen
    return {"name": name, "val_acc": vl_acc, "val_f1": vl_f1, "gap": gap,
            "min_recall": float(per_rec.min()), "score": score, **bench}


# ── util2.txt ──────────────────────────────────────────────────────
def export_util():
    try:
        r = subprocess.run([sys.executable, "-m", "pip", "freeze"],
                           capture_output=True, text=True, timeout=60)
        UTIL_SAVE_PATH.write_text(r.stdout, encoding="utf-8")
```

```
        print(f"  ✓ util2.txt → {UTIL_SAVE_PATH}")
    except Exception as e:
        print(f"  ⚠ util2.txt failed: {e}  (run: pip freeze > util2.txt)")


def main():
    seed_everything(SEED)
    tr_csv, vl_csv = OUTPUT_DIR / "train_split.csv", OUTPUT_DIR / "val_split.csv"
    if not tr_csv.exists() or not vl_csv.exists():
        print("  [ERROR] Run  python data_setup2.py  first."); sys.exit(1)
    train_df, val_df = pd.read_csv(tr_csv), pd.read_csv(vl_csv)

    results = []
    for cfg in VERSIONS:
        model, hist, eval_tf, pcount = train_version(cfg, train_df, val_df)
        bench = benchmark(model, eval_tf, val_df, pcount[0])
        res = evaluate_version(model, eval_tf, train_df, val_df, hist, cfg["name"], bench)
        results.append(res)
        del model; gc.collect()
        if DEVICE.type == "cuda": torch.cuda.empty_cache()

    # ── comparison table + chart ──────────────────────────────────────
    rdf = pd.DataFrame(results)
    rdf.to_csv(OUTPUT_DIR / "version_comparison.csv", index=False)
    print("\n" + "=" * 92 + "\n  VERSION COMPARISON\n" + "=" * 92)
    print(rdf[["name", "val_acc", "val_f1", "gap", "min_recall",
               "ms_per_image", "peak_vram_mb", "params_million", "score"]]
          .to_string(index=False))

    plt.figure(figsize=(11, 5))
    ax1 = plt.subplot(1, 2, 1)
    ax1.bar(rdf["name"], rdf["val_f1"]); ax1.set_title("Val macro-F1"); ax1.set_ylim(0, 1)
    ax2 = plt.subplot(1, 2, 2)
    ax2.bar(rdf["name"], rdf["ms_per_image"], color="orange")
    ax2.set_title("Inference ms/image")
    plt.tight_layout(); plt.savefig(FIGURES_DIR / "version_comparison.png", dpi=150)
    plt.close()

    # ── pick best & save as report2.pth (both locations) ──────────────
    best = rdf.sort_values("score", ascending=False).iloc[0]["name"]
    src = version_model_path(best)
    for dst in (BEST_MODEL_PATH, BEST_MODEL_PATH2):
        torch.save(torch.load(src, map_location="cpu", weights_only=False), dst)
    print(f"\n  BEST = {best}  → saved as report2.pth")
    print(f"     {BEST_MODEL_PATH}\n     {BEST_MODEL_PATH2}")

    export_util()

    # ── CNN interpretability on the best model ────────────────────────
    try:
        from visualize2 import run_all_visualizations
        run_all_visualizations(best_version=best, val_df=val_df)
    except Exception as e:
        print(f"  visualization skipped: {e}")

    print("\n" + "=" * 92 + "\n  ALL DONE).\n" + "=" * 92)


if __name__ == "__main__":
    main()
```